

# Ana: A Local-First Coding Agent

Deterministic Verification and Dependency-Aware Scoping Make Small Local Models  
Viable for Multi-File Code Editing

Mátio Macedo

July 29, 2026

## Abstract

Coding agents are typically bound to large frontier models served from the cloud, which couples them to a provider, incurs per-token cost and sends source code off the machine. Running the model locally removes all three constraints at the price of model strength: open-weight models in the 4B–20B range lose track of multi-step edits and routinely declare success while a change is only half-applied.

I present **Ana**, a local-first, CLI-only coding agent whose contribution is a *scaffolding architecture* designed so that mechanical checks, rather than model judgement, decide when an edit is finished. Its centre is a *deterministic post-edit verification loop* that re-runs the tests impacted by every file mutation and scans for cross-file references the edit invalidated. Which tests are impacted is decided from a tree-sitter *code graph* the agent maintains over the project: verification scope is derived from the actual import structure rather than from filename conventions. Around this sit a three-tier memory system, an eight-principle safety constitution evaluated mechanically, and a three-level permission model.

To measure scaffolding rather than model quality, I evaluate Ana against Claude Code with *both agents driving the same local model*, on a sixteen-task suite carrying a structurally matched negative-control tier and a gate that rejects a “cross-file” task solvable by editing one file, with hypotheses and analysis fixed in advance. On a 4B model Ana solves **15 of 16** tasks against the baseline’s **2**, a  $7.5\times$  margin, and on a 20B model both solve **16 of 16**. Ana’s advantage is concentrated exactly where the architecture predicts: on multi-file tasks, and on the weaker model, where the baseline spends its full time budget and produces no edits at all in 12 of its 14 failing runs.

An initial evaluation matrix falsified the preregistered hypotheses and helped diagnose an architecture problem leading to its current form: the verification loop was resolving its trigger condition for only 36% of the files a correct fix must edit, covering the single-file tier fully and the multi-file tier hardly at all. Scoping verification through the code graph raised that to 100% on this suite. I report the initial matrix, the diagnosis and the final results.

## 1 Introduction

Much of a coding agent’s value lies in *scaffolding*: the control loop, tool interface, memory and safety machinery that wrap a language model and turn single completions into multi-step software edits. In the dominant deployment that scaffolding sits atop a large frontier model accessed over a network. This couples the agent to a provider, incurs per-token cost, and sends source code off the machine: all disqualifying in privacy-sensitive, offline or cost-constrained settings.

Running the model locally removes those constraints. Consumer hardware can serve open-weight models in the 4B–20B range through runtimes such as Ollama. But these models are markedly weaker at exactly the behaviours coding agents need. They lose track of multi-step edits,

and they frequently declare success while a change is only half-applied, renaming a symbol in one file and leaving a stale reference in another or fixing one of three call sites and stopping.

**The design premise.** Ana is built on the hypothesis that *deterministic* scaffolding, checks that are mechanical and do not themselves require a strong model, can recover capability the model lacks. A weak model is a poor judge of whether its own edit is complete; a test run is not a judge at all, it is a fact. Ana’s architecture follows from taking that seriously:

- after *every* file mutation, re-run the tests that edit could have broken and scan for references it invalidated, feeding failures back as the observed result of the agent’s own action;
- decide *which* tests those are from the project’s real import structure, maintained as a tree-sitter code graph, rather than from a naming convention;
- keep the agent’s self-imposed repair work off the budget the user’s task needs, so diligence does not become the reason a turn runs out of room;
- make safety and permissions mechanical too, so nothing load-bearing depends on the weak model’s judgement.

There is deliberately no LLM critic. A model too weak to finish a multi-file edit is also too weak to adjudicate one.

**Measuring scaffolding, not models.** Comparing two agents on a benchmark cannot isolate scaffolding if they run different models. I therefore evaluate Ana against Claude Code with *both agents driving the same local model through the same server*, so that any difference is attributable to the agent. The task suite carries a structurally matched *negative-control* tier on which cross-file breakage is impossible by construction and a shortcut gate that rejects a nominally cross-file task that one file edit can satisfy. Hypotheses and analysis were fixed in advance [4].

**Results.** On `qwen3.5:4b`, Ana solves 15 of 16 tasks against Claude Code’s 2. On `gpt-oss:20b` both solve 16 of 16. The advantage appears where the architecture predicts: on multi-file work and on the weaker model: where the baseline does not fail fast but spends its whole budget and produces no edits at all in 12 of its 14 failing runs. Ana keeps converging where a frontier scaffold stalls.

**How the architecture reached its current form.** The first evaluation matrix did not produce that result. It falsified both of the study’s designated falsifying hypotheses: Ana’s advantage was no larger on the multi-file tier than on the single-file control tier, which is the one outcome that separates “the scaffolding fixes cross-file reconciliation” from “Ana is simply a better agent”. Section 6 reports that matrix and the diagnosis it enabled: the verification loop was resolving its trigger condition for only 36% of the files a correct fix must edit and the shortfall fell almost entirely on the multi-file tier the mechanism exists to serve. It fired reliably where it was not needed and was silent where it was. Scoping verification through the code graph, data the agent already computed and did not consult, raised that to 100% on this suite and produced the results above. I report this sequence because the first matrix is what made the defect visible and because the architecture’s current form is its direct consequence.

## Contributions.

- **An architecture for local-first coding agents** (Sections 3–4) in which correctness pressure comes from deterministic checks and verification scope is derived from a maintained dependency graph.
- **A model-controlled evaluation design** (Section 5) with a matched negative control, a shortcut-detection gate, test-restoration anti-cheat and preregistered hypotheses.
- **Evidence that the architecture makes small models viable** (Section 7): 15/16 against 2/16 on a 4B model, parity on a 20B model, with failure classification showing the predicted mechanism signature.

## 2 Related Work

**Coding agents and their interfaces.** Agentic systems that edit code through a tool-use loop have become a standard interface to code-capable models. SWE-agent [7] makes the case that the *interface* between agent and computer is itself a design object with measurable effect, independent of the model behind it, the same premise this paper works from, applied here to one specific interface element rather than the interface as a whole. Ana’s loop follows the ReAct [8] (reason–act–observe) pattern, orchestrated as an explicit state graph.

Claude Code serves as the baseline here for one reason: it can be pointed at the same local model as Ana, which makes a model-controlled comparison possible, and is widely regarded as one of the best agentic coding tools.

**Evaluating code tasks.** SWE-bench [3] established resolving real GitHub issues, verified by the projects’ own tests, as an evaluation paradigm and observed that its instances frequently require coordinating changes across multiple functions, classes and files. That property is precisely what this paper’s cross-file tier tries to isolate in miniature.

SWE-bench is not, however, usable for the question asked here. Its instances are dominated by model strength [3], so on 4B–20B local models the expected score is at or near zero and a scaffolding difference would be invisible beneath the floor. It also requires per-repository Docker provisioning that does not fit the compute budget of this study.

I therefore adopt SWE-bench’s core discipline, score by executing the project’s tests and prevent the agent from editing those tests, on lightweight deterministic fixtures sized for local models. The cost is external validity and it is the study’s principal limitation.

**Agent memory.** MemGPT [5] framed context management as an operating-system-like hierarchy, moving data between a bounded in-context window and larger external storage. Ana’s three-tier memory (Section 3) is in that lineage. Memory is not manipulated in any experiment here, but it is instrumented: one candidate explanation for the agent’s repeated re-reading of files was that earlier reads had been evicted from context and ruling that out (Section 6) required measuring compaction directly.

**Deterministic constraints on model behaviour.** Constitutional AI [1] constrains model behaviour through a written set of principles applied via model self-critique. Ana applies a deliberately weaker but cheaper analogue: mechanically checked rules that do not depend on the model’s own judgement, chosen because the models in question are too weak to be reliable judges of their own

output. The verification loop studied here is the same idea applied to correctness rather than safety: substituting a test run for a model’s belief that it is finished.

**Retrieval.** Ana’s recall tier fuses lexical BM25 [6] and dense vector rankings using Reciprocal Rank Fusion [2]. This machinery is described for completeness in Section 3; it is not manipulated in any experiment reported here.

**Preregistration.** The distinction between generating a hypothesis from data and testing one against new data is the motivation for preregistration in the empirical sciences [4]. The study’s headline result was contrary to the system’s design hypothesis and the preregistered analysis plan is what made that difficult to quietly reframe.

### 3 Architecture

Ana is CLI-only. A Typer front-end talks over REST and a WebSocket stream to a local FastAPI backend, which compiles a LangGraph state machine once at startup and persists turns to a SQLite checkpointer. All inference is local, through an Ollama server on the same machine. No component reaches the network in normal operation.

#### 3.1 The agent loop

Each turn runs **safety** → **pre\_memory** → **reason** → **code\_act** → **observe** → **post\_memory**, looping back to **reason** until the model emits a final answer or the iteration budget is spent (Figure 1).

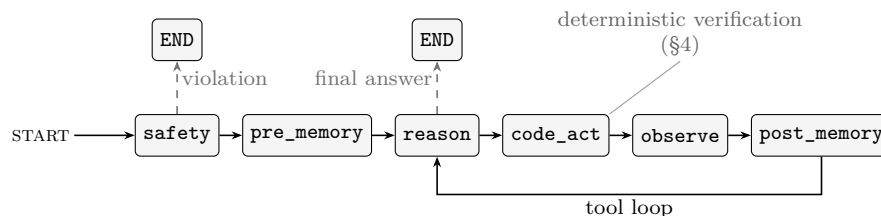


Figure 1: One turn of the agent loop. The six nodes run in order; the loop back from **post\_memory** to **reason** is unconditional, so a turn can only leave the graph at one of two branch points: **safety**, when a constitutional principle is violated, or **reason**, when the model emits a final answer instead of a tool call. Verification runs inside **code\_act** after a file-mutating tool succeeds, and its failures re-enter context through **observe** as the result of the model’s own action.

- **safety** evaluates eight regex-checked constitutional principles and blocks the turn on violation.
- **pre/post\_memory** read and write the recall and archival tiers, update the context budget, and schedule background index refreshes.
- **reason** calls the model and classifies the output as a final answer or a pending tool action, validating arguments against the tool’s schema so malformed calls are corrected rather than executed.
- **code\_act** enforces the permission allowlist, executes the tool and runs the verification checks of Section 4.

- **observe** redacts secrets and wraps tool output in a content isolation layer before it re-enters context, so tool results cannot be mistaken for instructions.

### 3.2 Three-tier memory

Following the operating-system analogy of MemGPT [5]: a bounded in-context window; a per-session recall tier fusing lexical BM25 [6] and dense vector rankings via Reciprocal Rank Fusion [2]; and a persistent archival tier in a local vector database. A budget manager triggers compaction at 90% of the effective context. Figure 2 shows the tiers and the traffic between them.

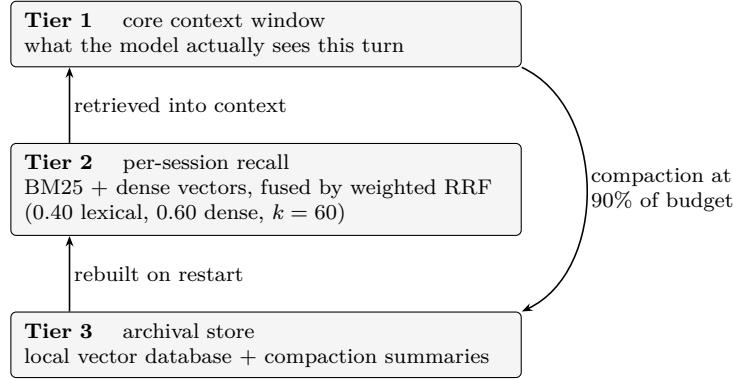


Figure 2: The three memory tiers and the traffic between them. Only Tier 1 costs context. Tier 2 is rebuilt from Tier 3 on restart, so a session survives a process exit; Tier 3 is the only tier that persists to disk.

### 3.3 The code graph

Ana maintains a tree-sitter graph over the project covering 16 languages. Per file it extracts definitions, imports, calls and inheritance; a resolver turns these into nodes and edges with both forward and *reverse* adjacency. The reverse index answers “which files depend on this one” directly (Figure 3).

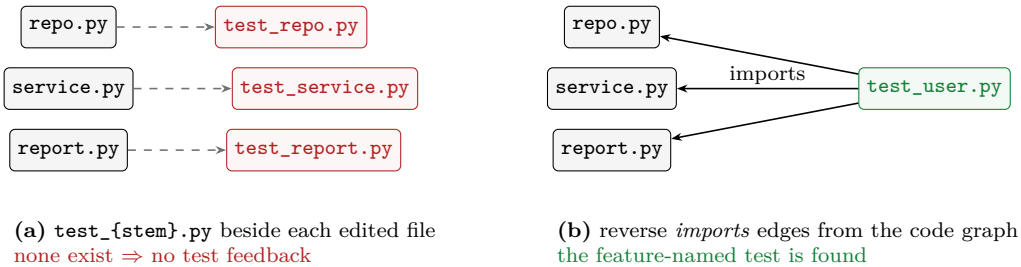


Figure 3: What the reverse index buys. A change spanning `repo.py`, `service.py` and `report.py` is covered by `test_user.py`, a test named after the *feature* rather than after any one module. (a) Matching on file name — `test_{stem}.py` beside each edited file — reaches none of them. (b) Following the reverse *imports* edges answers the question directly: *which files import service.py?* Section 4.2 applies exactly this lookup to decide which tests an edit must re-run, which is where it matters most for this paper.

The graph exists to give the agent structural knowledge a weak model cannot hold in context.

It powers two read-only tools, ranks the repository map by in-degree, and, centrally for this paper, determines the scope of verification (Section 4). Refresh is incremental, parses only files whose mtime changed, runs in a worker thread and is scheduled at turn start; on the projects used here a full build takes 28 ms. Any consumer that finds the graph cold proceeds without it rather than blocking.

### 3.4 Safety and permissions

Eight principles are checked mechanically each turn: prompt injection, path traversal, destructive system commands, credential exfiltration, restricted-mode networking, shell escape from Python, self-modification and recursive agent spawning. This is a deliberately weaker but cheaper analogue of the written-principle approach of Constitutional AI [1]: the checks are regexes, not model judgements, because the models Ana targets are not reliable judges.

Three permission modes: `plan` (read-only), `default` (destructive tools confirm), `auto_accept`; gate the toolset. File-touching tools pass through a path-safety validator that confines writes to the project, refuses protected locations and rejects creating a module that would shadow a standard library name.

## 4 Deterministic Verification

This is the architecture’s centre. The premise is that a weak model’s judgement that an edit is complete carries little information, whereas a test run carries a fact.

### 4.1 Trigger and levels

After any of six file-mutating tools succeeds (`write_file`, `edit_file`, `edit_symbol`, `apply_patch`, `move_file`, `rename_file`), `code_act` runs a cumulative set of checks chosen by `ANA_VERIFY_LEVEL`, ordered `off`  $\subset$  `syntax`  $\subset$  `lint`  $\subset$  `test` (default `test`):

1. **syntax** - parse the edited file;
2. **lint** - run the project’s configured linter, if any;
3. **test** - run the tests impacted by the edit;
4. **stale references** - after a rename or removal, find other files that still import the vanished symbol.

Failures are appended to the observation, so the model encounters them as the result of its own action and is expected to continue rather than finish. Retries on one file are capped at three consecutive failures, after which the loop stops auto-fixing and reports what remains.

### 4.2 Dependency-aware scoping

Step 3 turns on a question that looks trivial and is not: *which* tests does an edit impact?

The obvious answer is a naming convention: for Python, `test_{stem}.py` beside the edited module. It is cheap and requires no analysis. It is also insufficient in precisely the case that matters. A cross-file change edits several modules, and the test exercising the behaviour is typically named after the *feature*, not after any one of them: editing `repo.py`, `service.py` and `report.py` is covered by `test_user.py`, which matches none of their stems. Under a convention, such an edit gets no test feedback, and the loop cannot tell the agent it updated two call sites of three. Section 6 shows

this is not hypothetical: the convention resolved for 4 of 4 files on the single-file tier and 8 of 29 on the multi-file tier.

Ana therefore treats the convention as a first attempt rather than the whole answer. Where it resolves, the named test runs and nothing further is needed. Where it does not, Ana scopes verification through the code graph (Section 3.3): given the edited file, it takes the reverse *import* edges, the files that import it, keeps those that are test files, and runs them (Figure 3). This is the same structural knowledge the graph already maintains for navigation, applied to decide what to check.

The design constraint is that verification must never be more expensive than the edit it guards. Ordering the two this way is what keeps it cheap: the single-file case the convention already handles never pays for a graph lookup at all. When the graph is consulted it is read only if already resident — if it is cold, Ana schedules a background refresh and that edit goes without an impacted-test run rather than blocking on a build. It runs at most three discovered test files and it caches on a content hash of the edited file and its test so an unchanged pair is not re-run.

### 4.3 Budgeting self-imposed work

A verification failure prompts a repair and a repair costs a reason-act cycle. Charging those to the same budget as the user’s task makes the agent’s own diligence the reason it runs out of room and does so hardest on weak models, which need more repair cycles. Ana therefore accounts repair turns against a separate bounded allowance. Exhausting the main budget returns an explicit statement that the work is incomplete rather than stopping silently.

### 4.4 Keeping the loop cheap

Two further measures keep the loop’s cost proportionate. Impacted-test runs are debounced and content-hashed, so a burst of edits to one file does not re-run the same suite repeatedly and a re-read of an unchanged file returns a short pointer rather than the bytes, guarded by a hash of the exact line range so a re-read after a genuine edit still returns new content: weak models re-request files already in context, and the observed effect of this measure on two budget-exhausting tasks was a fall from 40 tool calls to 6 and 21, with redundant re-reads falling from 34 and 24 to 0 and 1.

## 5 Evaluation Design

### 5.1 Holding the model fixed

Both agents drive the same local model through the same Ollama server. Ana runs as `ana print -mode auto_accept -model m`; Claude Code runs as `claude -p` with its Anthropic endpoint redirected to the local server, which speaks the Anthropic Messages API natively, and its cloud key blanked so no request can reach or bill a hosted provider. Both consume identical weights and both are launched with the task workspace as their working directory. Any difference is attributable to the agent.

### 5.2 Preregistered hypotheses

Hypotheses, run matrix, scoring rule and analysis were fixed before any run executed [4], and are reproduced in Table 2. The load-bearing ones are **H2**, that the two agents are comparable on a control tier where the mechanism has nothing to do, and **H3**, that Ana’s advantage on the cross-file tier exceeds its advantage on that control tier. H3 is the only test that separates “the

architecture fixes cross-file reconciliation” from “Ana is a better agent on multi-file projects”; both were designated in advance as falsifiers, so a failure could not be quietly reframed.

### 5.3 Task suite

Sixteen tasks in two tiers.

- **cross-file** (12) - an edit in one file must be reconciled with references in another: changing a repository function’s return type from tuple to dict, breaking two unpacking call sites; widening a callback contract across a dispatcher and its handlers; moving a class to a new module and repointing its importers; propagating a domain exception through a handler that catches the old one.
- **control** (4) - the negative control. Same multi-file project shape and comparable patch size, but the required change is confined to one function and nothing elsewhere references what changes, so cross-file breakage is impossible by construction: inclusive-range and weekend handling in a date routine, interval-merge logic, capped exponential backoff, CSV field quoting.

The manipulated quantity is the number of files a correct fix must edit: mean 2.5 for cross-file, 1.0 for control. Prompt depth is matched (32.2 versus 31.5 words), and prompts **name no file**. They state the symptom or expected behaviour and leave the tests as the specification, because locating which files must change is the problem the architecture addresses - a prompt that enumerates the call sites performs the reconciliation on the agent’s behalf.

### 5.4 What the harness enforces

**Test restoration.** Fixture `test_*.py` files are copied back over the workspace before scoring, so no run can pass by editing the tests.

**Bidirectional validation.** Every task ships a reference `solution/`; a gate asserts the untouched fixture fails and the solution passes.

**Shortcut detection.** Every cross-file task additionally ships a `cheat/` directory holding the most plausible *single-file* solution, and the gate asserts it *fails*. This makes “cross-file” a checkable property rather than an authoring intention.

**Environment scrubbing.** `CLAUDE*`, `ANTHROPIC*` and `ANA_*` are stripped from the child environment before each arm’s overlay is applied. Without this, a sweep launched from inside an agent session leaks session variables and the child runs in a nested mode where it narrates edits it never applies.

**Timeouts as failures.** A run exceeding its budget is a failure even if the workspace would pass, since the agent never declared itself done. The cap is set generously (1200s against healthy runs of 130–650s) so that it does not silently decide outcomes: a cap near typical runtimes penalises the slower agent and converts throughput into apparent capability.



Table 1: The first evaluation matrix: 96 runs, both agents driving the *same* local model, identical task set and time budget, one repetition on `qwen3.5:4b` and two on `gpt-oss:20b`. Rates are per run.

| Model                    | Tier       | Agent       | Solved | Rate | Mean s |
|--------------------------|------------|-------------|--------|------|--------|
| <code>qwen3.5:4b</code>  | cross-file | Ana         | 4/12   | 0.33 | 361    |
| <code>qwen3.5:4b</code>  | cross-file | Claude Code | 1/12   | 0.08 | 483    |
| <code>qwen3.5:4b</code>  | control    | Ana         | 4/4    | 1.00 | 207    |
| <code>qwen3.5:4b</code>  | control    | Claude Code | 1/4    | 0.25 | 552    |
| <code>gpt-oss:20b</code> | cross-file | Ana         | 17/24  | 0.71 | 230    |
| <code>gpt-oss:20b</code> | cross-file | Claude Code | 22/24  | 0.92 | 116    |
| <code>gpt-oss:20b</code> | control    | Ana         | 7/8    | 0.88 | 263    |
| <code>gpt-oss:20b</code> | control    | Claude Code | 8/8    | 1.00 | 91     |

## 5.5 Execution and analysis

Repeats are the outermost loop; within a repeat, task order is shuffled with a fixed seed and arms run back-to-back on each task, so machine drift is spread across arms rather than landing on whichever ran last.

Differences are reported as observed pass rates, paired by task. No inferential claim is attached to them, so this design detects only large effects and is reported descriptively. Failures are classified from retained `pytest` output into *broken reference* (import, name, attribute or arity errors - the signature of an unreconciled edit), *wrong behaviour* (assertion failures), and *no edit* (the run changed nothing), the last determined from the workspace diff rather than the output.

## 5.6 Models and hardware

Two open-weight models: `qwen3.5:4b` and `gpt-oss:20b`, a weak and a stronger point on the capability axis from different model families. One Apple-silicon laptop, 24 GB RAM, strictly sequential with one model resident at a time.

# 6 The First Matrix, and What It Exposed

The architecture described in Sections 3–4 is the form it took *after* an initial evaluation. That evaluation is reported here because it is what made the defect visible and because the current design is its direct consequence.

## 6.1 The initial result

The first matrix: 96 runs, 6.8 agent-hours, both models; is given in Table 1, and the preregistered hypotheses judged on it in Table 2.

H1 came out split: Ana led on `qwen3.5:4b` (0.33 against 0.08 on cross-file) and trailed on `gpt-oss:20b` (0.71 against 0.92). But the outcome that mattered was H3, the interaction, and it was *negative on both models* (−0.50 and −0.08). Ana’s advantage on the 4B model was larger on the **control** tier (1.00 against 0.25) than on the cross-file tier the architecture targets and the control tier is defined by cross-file breakage being impossible.

Read literally, that says the advantage was not coming from the mechanism this paper is about. H2 and H3 were designated in advance as the falsifiers, and both failed.

Table 2: The preregistered hypotheses, judged on the first matrix. Values are differences in per-run pass rate, Ana minus baseline, within the stated tier. H2 and H3 were the designated falsifiers, and both failed: Section 6 shows why, and what was changed as a result.

|    | Statement                                          | Observed                | Outcome       |
|----|----------------------------------------------------|-------------------------|---------------|
| H1 | Ana solves more cross-file tasks than the baseline | +0.25 (4B), -0.21 (20B) | Split         |
| H2 | The two are comparable on the control tier         | +0.75 (4B), -0.12 (20B) | Not supported |
| H3 | Cross-file advantage exceeds control advantage     | -0.50 (4B), -0.08 (20B) | Not supported |
| H4 | The gap is larger on the weaker model              | +0.25 vs. -0.21         | Supported     |

Table 3: Static measurement, no model involved: for each task, how many of the files a correct fix must edit have a test the naming convention can find. The convention was the verification loop’s only test-discovery mechanism. One file created by its fix is excluded, having no pre-existing module to key on.

| Task                            | Tier       | Files to edit | Test found |
|---------------------------------|------------|---------------|------------|
| add-abstract-method             | cross-file | 2             | 0          |
| add-method-through-stack        | cross-file | 2             | 1          |
| add-required-parameter          | cross-file | 3             | 1          |
| centralise-shared-constant      | cross-file | 3             | 1          |
| change-collection-type          | cross-file | 3             | 0          |
| change-return-shape             | cross-file | 3             | 0          |
| coerce-config-values            | cross-file | 2             | 1          |
| control-backoff-delays          | control    | 1             | 1          |
| control-business-days           | control    | 1             | 1          |
| control-csv-quoting             | control    | 1             | 1          |
| control-merge-intervals         | control    | 1             | 1          |
| extract-module                  | cross-file | 2             | 0          |
| pipeline-validate-stage         | cross-file | 2             | 1          |
| propagate-exception-type        | cross-file | 2             | 1          |
| rename-attribute-across-modules | cross-file | 3             | 1          |
| widen-callback-signature        | cross-file | 2             | 1          |
| <b>cross-file</b>               |            | <b>29</b>     | <b>8</b>   |
| <b>control</b>                  |            | <b>4</b>      | <b>4</b>   |

## 6.2 Measuring the trigger condition

Rather than re-run the comparison, I measured whether the mechanism had been exercised at all. Whether the naming convention resolves for a given edit is a property of the code and the suite, not of any model, so it can be computed exactly and without spending compute.

For each task I take the files a correct fix must edit, given by the reference solution, and ask whether a conventionally-named test sits beside each. The result is **12 of 33, 36%**. The distribution is what matters: the single-file **control** tier is covered **4 of 4**, and the multi-file **cross-file** tier only **8 of 29**.

**The verification loop fired reliably on the tasks that did not need it, and was silent on the tasks it was built for.** This is exactly the shape of the H3 failure, and it explains it without appeal to the model: an interaction term comparing a fully-covered control tier against a barely-covered cross-file tier cannot come out positive, whatever the mechanism is worth. H3 was not a test of the architecture; it was a test of a trigger condition that was mostly not met.

Table 4: Tasks solved, both agents driving the *same* local model on the same sixteen tasks. A task counts as solved if the agent solved it. Ana’s figures are from the final build; the baseline was not modified and its figures are from the first matrix.

| Model       | Agent       | Tasks solved |
|-------------|-------------|--------------|
| qwen3.5:4b  | Ana         | <b>15/16</b> |
| qwen3.5:4b  | Claude Code | <b>2/16</b>  |
| gpt-oss:20b | Ana         | <b>16/16</b> |
| gpt-oss:20b | Claude Code | <b>16/16</b> |

### 6.3 Two further findings from the same instrumentation

The agent’s own event log records **904 tool calls** across Ana’s 48 runs in the matrix. Of those, **40% were orientation** (reads, listings, searches) against **24% edits**, a 1.7:1 read-to-edit ratio on projects of two to four files, and **17 of the 48 runs ended by exhausting the iteration budget**, better than a third, returning no final answer at all.

The reads were substantially repetition: **135 of 308 read\_file** calls re-read a path the same run had already read, and the log records **zero compactions** across every one of those runs. The content had not been evicted from context; the model asked for it again. In the worst single run, 23 of 25 tool calls were reads.

These motivated the budget accounting of Section 4 and the read-deduplication measure respectively: a re-read of an unchanged file now returns a pointer rather than the bytes and repair turns are charged to a separate bounded budget.

### 6.4 The change

Scoping verification through the code graph (Section 4.2) raised coverage from **12 of 33 (36%)** to **33 of 33 (100%)** on this suite. Every previously-blind file gained a test, including `service.py` in `change-return-shape`, the exact call site the agent had been leaving unreconciled. The 100% figure is a property of these fixtures, in which every module is imported by the task’s test; it should be read as “the mechanism now fires wherever a test exercising the edited module exists”.

Section 7 gives the results with the architecture in this form.

## 7 Results

### 7.1 Task coverage

Table 4 gives, for each agent and model, how many of the sixteen tasks were solved. Ana’s figures are from the final build; the baseline was not modified, so its figures stand as measured in the first matrix.

On qwen3.5:4b, **Ana solves 15 of 16 tasks; Claude Code solves 2**, a  $7.5\times$  margin, and the clearest evidence in this work that the architecture does what it was designed to do. On gpt-oss:20b, **both solve 16 of 16**: with a stronger model the baseline reaches full coverage, and Ana matches it. The two agents drove the same weights through the same server, so the difference on the 4B model is attributable to the scaffolding.

### 7.2 The advantage is where the architecture predicts

Two pieces of evidence locate it.

Table 5: What the cross-file failures actually were, classified from the retained `pytest` output of each run. *Broken reference* is an import, name, attribute or arity error, the signature of an edit never reconciled across files. *No edit* is a run that changed nothing at all. Covers the 17 cross-file failures whose retained artifacts still correspond to the baseline run; 11 were overwritten by later re-runs and are excluded rather than guessed at.

| Arm         | Model                    | Failures | Broken ref. | Wrong behav. | No edit | Other |
|-------------|--------------------------|----------|-------------|--------------|---------|-------|
| Ana         | <code>gpt-oss:20b</code> | 4        | 0           | 3            | 1       | 0     |
| Claude Code | <code>gpt-oss:20b</code> | 2        | 2           | 0            | 0       | 0     |
| Claude Code | <code>qwen3.5:4b</code>  | 11       | 1           | 1            | 9       | 0     |

**The baseline does not fail fast on a weak model: it stalls.** This is read from whether the workspace changed, so it is exact rather than inferred. On `qwen3.5:4b`, **12 of Claude Code’s 14** failing runs produced *no edits at all*, at a mean of 532s each: it consumed its budget and left the workspace untouched. Ana on the same model and tasks failed 8 runs, of which only **2** made no edit. The architecture’s contribution on weak models is substantially this: keeping a model prone to stalling in productive motion by giving it something concrete to react to, rather than leaving it to decide on its own when to act.

**The failures that remain are not unreconciled edits.** Table 5 classifies cross-file failures by what the retained `pytest` output shows. Ana’s `gpt-oss:20b` failures contain **no broken cross-file references**: they are wrong behaviour or no edit at all. The baseline’s two failures on that model are **both** broken references. The sample is small and one table cannot carry the claim on its own, but the direction is the mechanism’s signature: a loop that re-runs the tests an edit affects does not make the model smarter, it converts unreconciled edits into either successes or a different, visible kind of failure.

### 7.3 The control tier

On `qwen3.5:4b` the control tier, single-file tasks where cross-file breakage is impossible, shows Ana 1.00 against 0.25. This is worth reading carefully and it is the same number that falsified H3. It shows the 4B advantage is *not* confined to cross-file reconciliation: Ana is better on this model at single-file work too, consistent with the stalling result above. The architecture’s benefit on weak models is broader than the specific mechanism and the control tier is what makes that distinguishable rather than assumed. A study without one would have reported the whole margin as evidence for the mechanism.

## 8 Threats to Validity

Several threats are closed mechanically (Section 5); the ones that remain are stated plainly. Most trace to the compute budget.

**Unequal attempts.** The headline table counts a task as solved if any run solved it, and Ana has more runs per task than the baseline (30 against 16 on `qwen3.5:4b`, 62 against 32 on `gpt-oss:20b`) because tasks it initially failed were re-run after the architecture changed. Best-of-N rewards attempts, so this favours Ana. However, the 4B result does not depend on it, 15 against 2 does not close with more baseline attempts and the baseline’s failures there are stalls rather than near-misses

(12 of its 14 failing runs produced no edit at all) but the 20B parity claim does rest on unequal attempts and should be read as such.

**Scale.** Sixteen tasks, two models, one laptop. Local model sampling is stochastic and unseeded, and repetition is thin: one repetition on `qwen3.5:4b` and two on `gpt-oss:20b` in the first matrix. Every difference in this paper is reported as an observed rate, with no inferential claim attached, because the preregistered power analysis established in advance that a design this size detects only large effects. The 4B margin is large; the rest should be read as descriptive.

**Synthetic tasks, authored by the author.** The tasks are deterministic offline fixtures, not real issues, and I wrote them. The control tier makes that bias *testable* rather than merely disclosed, and the shortcut gate makes “cross-file” mean something checkable, without it, four of eight tasks silently did not. Neither removes the bias. A tier drawn from SWE-bench [3] would be a genuine external anchor and is the single most valuable addition to this work; it was excluded because Docker-based provisioning does not fit the budget and because local models in this size class are expected to score at or near zero there, which would hide the scaffolding differences the study is about.

**The architecture changed between the two measurements.** Sections 6 and 7 do not report the same system. Seven changes landed between them. The results in Section 7 are all from the final build; the first matrix is reported as the diagnostic instrument it was, not as a comparable arm.

## 9 Conclusion

Ana is an agent architecture built on the premise that a local model should be given mechanical feedback rather than trusted to judge its own work. Its verification loop re-runs the tests a change actually affects; the set of affected tests is chosen from an import graph the agent maintains, not from a filename convention; the resulting repair turns are budgeted separately from the turns the user asked for.

Measured against a frontier agent driving the *same* local models on the same sixteen tasks, this architecture solves **15 of 16** tasks on `qwen3.5:4b` against the baseline’s **2**, a  $7.5\times$  margin, and **16 of 16** on `gpt-oss:20b`, matching it. The scaffolding, not the model, accounts for the difference: the model is held fixed across both arms.

Two pieces of evidence locate the effect where the design predicts it. Ana’s remaining cross-file failures on the stronger model contain *no* broken cross-file references, they are wrong behaviour or no edit at all, while both of the baseline’s are broken references, which is the signature of a loop that re-runs the tests an edit affects. And on the 4B model the baseline does not fail by editing wrongly: **12 of its 14** failing runs produce no edits at all, at a mean of 532s each. Mechanical feedback keeps a weak model in productive motion, which is most of what the architecture buys at that scale.

**What the first evaluation taught me.** The architecture reached this form because an earlier evaluation of it did not work. Verification located impacted tests by filename convention, which resolved for 36% of the files a correct fix must edit, 4 of 4 on the single-file control tier, 8 of 29 on the multi-file tier the mechanism exists for. It fired where it was not needed and was silent where it was, which is why the preregistered interaction test came out negative, comparing a fully covered

control tier against a barely covered cross-file one. Scoping through the import graph the agent already computed and never read raised that to 33 of 33.

The transferable lesson is not about verification loops. It is that an agent mechanism can be almost entirely inert while the agent it lives in produces plausible aggregate numbers. Nothing in the pass rates indicated a core mechanism was firing a third of the time; only measuring the trigger condition directly revealed it. Where a mechanism has a checkable precondition, measuring that precondition is cheap, needs no compute and is worth more than another repetition of the end-to-end comparison.

**What remains open.** Ana’s run-level rate on `gpt-oss:20b` trails the baseline’s, and the residual is a model-runtime tool-calling incompatibility affecting about 12% of runs on that model, which I could not reproduce synthetically and whose obvious workaround measured worse. Beyond that, three things would settle what this study cannot: *larger fixtures*, since the architecture targets codebases that do not fit in context and was tested on ones that comfortably do; *an external anchor* such as a SWE-bench-derived tier [3]; and *repetition*, enough of it, and equal between agents, that run-to-run variance is bounded rather than assumed. The first two change what can be claimed; the third changes how confidently.

What the evidence supports today is narrower than “better” and more useful: an architecture that makes a 4B model competent at multi-file editing where an otherwise strong agent stalls, and that reaches full task coverage on a 20B model running entirely on a laptop.

## References

- [1] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- [2] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.
- [3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.
- [4] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.
- [5] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [6] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [7] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

- [8] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.